

Full-Stack Engineering Internship

Your First 3 Weeks

Foundations & Design — Weeks 1 to 3

Welcome — How These 3 Weeks Work

You're one of four interns starting together. For these first 3 weeks, you won't be writing framework code yet — no Express, no FastAPI, no Django. Instead, you'll build the thinking that sits underneath any framework: how the web actually works, how to design an API, how to model data, how to reason about security and reliability, and how to work directly with LLMs.

Why start this way? Frontend implementation is the part AI tooling accelerates the most today — it's rarely what separates a strong engineer from an AI-assisted app. The skills that actually differentiate you are systems thinking, data modeling, API design, and reasoning through tradeoffs. That's what these 3 weeks are for.

You'll get one deliberate touchpoint on the frontend side (Week 1, Day 2) — enough to read a network request and reason about the client-server boundary. That's it for now; framework-level frontend work comes later, once you've specialized.

One thing to keep in mind throughout: everything you produce — specs, diagrams, plans — should be something you can defend, not just something you turned in. Mentors will cross-question your choices. That's the primary way your work gets evaluated here, more than whether the deliverable exists.

Daily & Weekly Rhythm

These run alongside the day-by-day plan below, every day, for all 3 weeks:

- ☐ Daily problem-solving: solve 1 problem-solving question per day (LeetCode or a similar platform). Keep it time-boxed so it doesn't eat into your core work for the day. Report what you attempted and the outcome to your individual lead — every day.
- ☐ Daily standup: at the end of each day, post a short written update — what you did, any issues or blockers you're facing, and any questions you have. Goes to your individual lead (or your team's shared channel).
- ☐ Discussion meetings: twice a week, you'll meet for a session where you're cross-questioned on what you've worked on since the last meeting. This is separate from the Friday checkpoint — a more frequent pulse-check on your thinking, not a repeat of the deeper Friday review.

Daily Standup Template

Keep it short — a few lines per section is enough. Post it before you log off each day.

What I did today:

Issues / blockers I'm facing:

Questions I have:

Week 1 — Web Systems Basics + Light Design

Day 1 — How the web actually works

- ☐ Client-server model, HTTP request/response cycle, headers, status codes
- ☐ Know the idempotent vs. non-idempotent methods (GET/PUT/DELETE vs. POST) — you'll need this reasoning later when designing APIs
- ☐ Cover status code families (2xx/3xx/4xx/5xx), not just 200/404
- ☐ Note the headers that matter for APIs specifically: Content-Type, Authorization, Cache-Control
- ☐ Set up your tools: Postman/curl, OpenAPI editor, ERD tool, sandbox DB access
- ☐ Git workflow refresher — branching, clear commit messages, opening a PR. You'll be doing this daily from here on
- ☐ Read: [architecture overview], [style guide] — flag anything that seems to conflict with what you already assumed; bring it to your lead

Day 2 — Frontend/backend boundary

- ☐ Pick a real site with a login or search flow — richer network activity to study than a static page
- ☐ Open devtools and inspect the network tab — what requests fire, what's client state vs. server data, what a JSON payload looks like end to end
- ☐ Trace one full request start to finish: what triggered it, what was sent, what came back, what the page did with the response
- ☐ Be ready to discuss what belongs client-side vs. server-side, and why that boundary matters for security and performance
- ☐ Write down one concrete way a client-only validation check could be exploited if there were no server-side check
- ☐ No framework, no UI building today — this is about reading and reasoning, not building

Day 3 — Intro API design

- ☐ Design a simple CRUD spec: resource design, status codes, basic request/response shape (OpenAPI)
- ☐ Name resources deliberately — nouns, not verbs, consistent singular/plural
- ☐ Give every endpoint a status code for every case up front — not-found, invalid input, unauthorized — not just the happy path
- ☐ Write at least one example request/response payload per endpoint, not just the schema — this catches shape problems schemas alone miss
- ☐ This is a lighter pass — keep it to 3–5 endpoints; the deep version comes in Week 2

Day 4 — Systems thinking / architecture basics

- ☐ Given a one-paragraph product brief, sketch the system: client, server, DB, external services
- ☐ Every endpoint from your Day 3 spec should map to something in this diagram
- ☐ Label data flow direction, not just boxes and lines
- ☐ Note where you'd add a queue, cache, or background job if this had to scale, even if it doesn't need one yet
- ☐ Think through: monolith vs. services, sync vs. async, where an AI feature would sit in your diagram
- ☐ Deliverable: an architecture diagram, informed by the API spec you designed on Day 3

Day 5 — Checkpoint

- ☐ Present your architecture diagram and intro API spec
- ☐ Walk through your diagram as if narrating a real request, start to finish
- ☐ Be ready to name one decision you'd make differently in a real production system, and why you didn't here
- ☐ Come with a question of your own — something you're still unsure about from the week

Week 2 — Harder Foundations & Design

Day 1 — Databases

- ☐ You'll be given your own project domain for this and the following exercises
- ☐ Identify entities and relationships before you touch SQL — one-to-many vs. many-to-many changes your schema significantly
- ☐ Decide your primary key strategy (auto-increment vs. UUID) and be able to justify it
- ☐ Build an ER diagram for your domain
- ☐ Hands-on raw SQL: DDL with real constraints (NOT NULL, foreign keys, unique — not bare tables), DML (seed + query), joins, aggregates
- ☐ Write at least one multi-table join query and one aggregate query (COUNT/SUM/GROUP BY) against your own seed data
- ☐ Migrations explained conceptually — think through what adding a required column to a table with existing data would break

Day 2 — Deeper API design

- ☐ Design a multi-resource API with relationships, pagination, filtering, and a versioning strategy — a full OpenAPI spec, built on your Day 1 schema
- ☐ Every relationship in your Day 1 ER diagram should show up here — nested resources, foreign key references, or both
- ☐ Design pagination for real — cursor-based or offset-based — and know why you picked one
- ☐ Add at least 2 filter/query parameters to a list endpoint, and define what happens with invalid combinations
- ☐ Pick a versioning approach and name the tradeoff you're accepting
- ☐ Revisit your Week 1, Day 3 spec — what would you change now that you know more?

Day 3 — Auth & security

- ☐ Design a JWT/session auth flow as a sequence diagram — cover both the login flow and a subsequent authenticated request, not just login
- ☐ Decide where the token lives (header vs. cookie) and note the tradeoff of your choice
- ☐ Think through token expiry and refresh — what happens when a token expires mid-session?
- ☐ Audit 4–5 vulnerable code snippets against the OWASP Top 10 — name the specific category and the one-line fix for each, not just "this is bad"
- ☐ Discuss how the threat model shifts on an AI-touching endpoint — what's the prompt-injection equivalent of SQL injection?

Day 4 — Testing & reliability design

- ☐ Write a test plan: cases, edge cases, what "done" means — cover at least one happy-path, one invalid-input, and one authorization-failure case per endpoint
- ☐ Think about what "done" means beyond "the tests pass" — what would you want to know before merging this to production?
- ☐ Write an error-handling and structured-logging strategy as a design doc — aim for production realism, not demo quality
- ☐ Decide what gets retried automatically vs. surfaced to the caller, and why
- ☐ Define what fields every log line should include (request ID, user ID, timestamp) so issues are traceable later

Day 5 — Checkpoint

- ☐ Present your full design package: API spec + ER diagram + auth flow + test plan + reliability design
- ☐ Be ready to trace a single request end-to-end across all five artifacts — from the API call to the DB query to the log line it produces
- ☐ Identify the weakest part of your own design package and be ready to say why

Week 3 — Prompt Engineering & AI Integration

Day 1 — LLM API fundamentals

- ☐ Provider API structure: system vs. user prompt, message roles, streaming
- ☐ Tokens, cost, and latency tradeoffs — estimate the cost of one call, then of 10,000 calls a day
- ☐ Call the LLM API directly via curl/Postman and inspect the raw request/response, including metadata (token counts, stop reason, model version), not just the text
- ☐ Send the same prompt with and without a system prompt and compare the difference
- ☐ Try a streaming response if your provider supports it, and note what changes about handling it in real code
- ☐ Deliberately send a malformed request (missing field, bad auth) and look at the error you get back

Day 2 — Prompt engineering, core techniques

- ☐ Clear and specific instructions — vague vs. precise, side by side
- ☐ Few-shot prompting — positive and negative examples
- ☐ Chain-of-thought reasoning
- ☐ Structured output (JSON/XML) and why it matters for parsing — try actually parsing the response as JSON in code and see what breaks when it doesn't come back clean
- ☐ Role/persona framing, system prompts vs. inline instructions
- ☐ For each technique, run the same base task with and without it — notice which ones actually move the needle for your task and which don't
- ☐ Find one case where adding more instructions made the output worse — over-specifying is a real failure mode too

Day 3 — Prompt engineering lab

- ☐ Hands-on: write and test prompts for classification, extraction, and summarization — use the same handful of test inputs across all three so quality is comparable
- ☐ Compare zero-shot vs. few-shot output quality
- ☐ For extraction specifically, test what happens when the requested information isn't actually in the input — does the model say so, or guess?
- ☐ Try a basic prompt-injection attempt against your own prompt (e.g. "ignore previous instructions" embedded in the input) and see what happens
- ☐ Discuss failure modes: hallucination, inconsistent formatting, prompt injection awareness — keep a specific log of how outputs failed, not vague notes

Day 4 — Tool use / function calling

- ☐ Learn function/tool calling as a pattern for structured, reliable outputs — and the difference between the model calling a tool vs. just producing text that looks like one
- ☐ Design exercise: spec the integration contract into your Week 2 API
- ☐ Your contract should specify what happens when the LLM's response doesn't match the expected schema
- ☐ Design your error-handling approach: a justified timeout value, rate limits, malformed/empty responses — concrete for your use case, not abstract

Day 5 — Prompt eval + final checkpoint

- ☐ Build a prompt eval: test inputs + expected outputs + hit rate — include at least one deliberately tricky/edge-case input, not only easy ones
- ☐ Define your hit-rate criteria precisely before you run the eval — "close enough" isn't measurable
- ☐ Refine based on results; note what you changed and whether it actually helped
- ☐ Demo: your prompt, its eval results, and your Day 4 integration contract
- ☐ Retro: what clicked, what was confusing, what you'd do differently with more time

What's Next

Starting Week 4, you'll move into a specialized stack — Python or Node.js — where the designs you've built over these 3 weeks get implemented for real. How you approach these first 3 weeks — the rigor in your specs, how you handle cross-questioning, how you reason through tradeoffs — feeds into how that split gets decided.

Good luck — build things you can defend.